

Artificial Neural Network (ANN) Guide

In-Depth Theory & Detailed Line-by-Line Code Explanation (MachineLearning.ipynb)

Part 1: Detailed Neural Network Theory (Explained Simply)

An **Artificial Neural Network (ANN)** is a machine learning model inspired by the structure of biological neurons in the human brain. It works as a collaborative stack of computational layers that learn patterns from data by continually adjusting numerical weights.

High-Level Network Architecture

[Input Layer] → [Hidden Layer 1] → [Hidden Layer 2] → [Output Layer]

Features enter through the Input Layer, pass through Hidden Layers for pattern extraction, and produce a final class probability at the Output Layer.

1. Network Layer Breakdown

- **Input Layer:** Receives raw feature attributes (e.g., *Glucose*, *Blood Pressure*, *BMI*, *Age*). Each input node maps directly to one feature column. No mathematical transformations take place in this layer.
- **Hidden Layers:** The core analytical engine of the network. Every neuron inside a hidden layer takes inputs from the previous layer, computes a weighted linear combination, adds a bias term, and applies a non-linear activation function.
$$z = (x_1 \times w_1) + (x_2 \times w_2) + \dots + (x_n \times w_n) + b$$
 - **Weights (w):** Coefficients determining feature relative importance.
 - **Bias (b):** Offset parameter shifting the activation threshold.
- **Output Layer:** Produces the final model prediction. For binary classification (e.g., Diabetic vs. Non-Diabetic), a single neuron yields a continuous probability score between 0.0 (0%) and 1.0 (100%).

2. Activation Functions

Without activation functions, a neural network simplifies into a basic linear regression model regardless of depth. Activation functions introduce **non-linearity**, enabling the network to fit complex, curved decision boundaries.

Activation Function	Layer Location	Formula & Mathematical Function
ReLU (Rectified Linear Unit)	Hidden Layers	$f(x) = \max(0, x)$ Outputs 0 for negative values, passes positive values unchanged. Accelerates gradient descent training.
Sigmoid	Output Layer	$\sigma(z) = 1 / (1 + e^{-z})$ Squeezes continuous outputs into bounded probability values [0, 1]. Ideal for binary outcomes.

3. How Neural Networks Learn: The 3-Step Cycle

1. **Forward Propagation:** Feature inputs pass forward through the layers to generate a predicted probability ($\hat{y}$).

2. **Loss Calculation (Binary Cross-Entropy):** Measures error between predicted score $\hat{y}$ and ground-truth binary outcome y :

$$Loss = - [y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

3. **Backpropagation & Optimization (Adam):** Using the calculus Chain Rule, backpropagation computes partial derivatives ($\partial Loss / \partial w$) to determine each weight's error contribution. The **Adam Optimizer** updates all weights backward across mini-batches to minimize overall loss over multiple **Epochs**.

Part 2: Detailed Line-by-Line Code Breakdown

Below is the complete Keras Artificial Neural Network implementation from `MachineLearning.ipynb` alongside a step-by-step code explanation:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Load dataset
diabetes_data = pd.read_csv('/content/drive/MyDrive/diabetes dataset for training.csv')

# Separate features (X) and target label (y)
X = diabetes_data.drop('Outcome', axis=1)
y = diabetes_data['Outcome']

# Split data into training (80%) and testing (20%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Feature Standardization
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Build Neural Network Architecture
model = Sequential()
model.add(Dense(64, activation='relu', input_shape=(X_train.shape[1],)))
model.add(Dense(64, activation='relu'))
model.add(Dense(1, activation='sigmoid'))

# Compile Model
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])

# Train Network
model.fit(X_train, y_train, epochs=100, batch_size=32, verbose=1)

# Evaluate and Predict
_, accuracy = model.evaluate(X_test, y_test, verbose=0)
print('Accuracy:', accuracy)

new_data = pd.DataFrame([[8, 183, 64, 0, 0, 23.3, 0.672, 32]], columns=X.columns)
new_data_scaled = scaler.transform(new_data)
```

```
prediction = model.predict(new_data_scaled)
print('Prediction:', prediction)
```

Step-by-Step Code Analysis

- `import pandas as pd` : Imports Pandas library for structured tabular dataset handling.
- `from tensorflow.keras.models import Sequential` : Imports the Sequential container to stack neural network layers sequentially.
- `from tensorflow.keras.layers import Dense` : Imports fully connected (Dense) layers where every neuron connects to all nodes in the preceding layer.
- `diabetes_data = pd.read_csv(...)` : Reads the training CSV dataset into a Pandas DataFrame.
- `X = diabetes_data.drop('Outcome', axis=1)` : Drops the target column so `X` holds only the 8 medical feature columns.
- `y = diabetes_data['Outcome']` : Isolates target column containing binary labels (0 = Non-diabetic, 1 = Diabetic).
- `train_test_split(..., test_size=0.2, random_state=42)` : Splits dataset into 80% training set and 20% test set. `random_state=42` locks dataset shuffling for identical splits across runs.
- `scaler = StandardScaler()` : Initializes Z-score standardizer ($\mu = 0, \sigma = 1$).
- `X_train = scaler.fit_transform(X_train)` : Computes mean and standard deviation on training data and transforms feature scale. Essential so large ranges (e.g. *Insulin: 0-800*) do not overwhelm small ranges (e.g. *Pedigree: 0.08-2.4*).
- `X_test = scaler.transform(X_test)` : Scales test features using training distribution statistics to prevent test data leakage.
- `model = Sequential()` : Instantiates empty linear layer container.
- `model.add(Dense(64, activation='relu', input_shape=(X_train.shape[1],)))` : Adds **First Hidden Layer** with 64 neurons and ReLU activation. `input_shape` specifies the 8 input feature dimensions.
- `model.add(Dense(64, activation='relu'))` : Adds **Second Hidden Layer** with 64 neurons and ReLU activation for deep pattern extraction.
- `model.add(Dense(1, activation='sigmoid'))` : Adds **Output Layer** with 1 single neuron and Sigmoid activation function to output binary class probabilities.
- `model.compile(...)` : Configures training settings using Binary Cross-Entropy loss function, Adam adaptive optimizer, and accuracy evaluation metric.
- `model.fit(..., epochs=100, batch_size=32, verbose=1)` : Executes model training over 100 passes (epochs) through full dataset in mini-batches of 32 samples.
- `model.evaluate(X_test, y_test, verbose=0)` : Computes test accuracy on unseen test set (~74.68% accuracy).
- `new_data_scaled = scaler.transform(new_data)` : Scales new patient metrics using original scaler parameters.
- `prediction = model.predict(new_data_scaled)` : Evaluates network to output patient diabetes probability score (e.g. `[[0.744]]` = 74.4% probability of diabetes).